

Decentralized Edge Workload Forecasting With Gossip Learning

Alessandro Tundo¹, Federica Filippini, Francesco Regonesi, Michele Ciavotta²,
and Marco Savi¹, *Member, IEEE*

Abstract—Edge computing has emerged as a crucial paradigm for addressing the growing demands of interconnected devices and large-scale mobile applications by relocating computation and storage services closer to end-users. Edge workloads are inherently volatile and challenging to forecast due to their dependence on factors such as human mobility patterns and geographically-distributed infrastructure, combined with the dynamic nature of edge nodes. Traditional centralized approaches to workload forecasting are inadequate in the context of decentralized and failure-prone edge environments. To address this challenge, this paper investigates workload forecasting using Gossip Learning (GL), an asynchronous peer-to-peer learning protocol. GL allows for the training of forecasting models in a fully-decentralized manner, thereby mitigating single point of failure risks and enhancing overall system robustness. We extended the original protocol across multiple dimensions to improve convergence, reduce communication overhead, and enhance resilience to failures. We evaluated the proposed approach through extensive simulations; the obtained results demonstrate its effectiveness with respect to classical methods, rendering it a promising solution to enhance load balancing and task offloading strategies at the edge, thereby ensuring Quality-of-Service (QoS) and reducing Service Level Agreement (SLA) violations.

Index Terms—Edge computing, workload forecasting, gossip learning, function-as-a-service, machine learning.

I. INTRODUCTION

EDGE computing has rapidly evolved to complement and extend cloud computing, addressing the increasing demands of interconnected devices and large-scale mobile applications. This paradigm shifts computation and storage services closer to users by positioning them at the network

edge, thereby meeting the requirements for ultra-low latency, location awareness, optimized bandwidth allocation, reduced execution costs, and lower energy consumption [1]. However, edge computing also introduces unique challenges due to its dynamic and geographically-dispersed nature. Edge workloads are particularly volatile and difficult to predict as they depend on several factors like the interaction between human mobility patterns and a geo-distributed computing infrastructure [2] and the particular application scenario (e.g., IoT data ingestion and processing [3], [4], [5], Machine Learning (ML) model inference and training [6], [7], [8], or virtual network functions (VNFs) execution [9], [10], [11]). Moreover, edge nodes are often heterogeneous, with limited capacity, and tend to join and leave the network due to unpredictable link and node failures. Consequently, defining optimal resource allocation and task offloading policies to guarantee Quality-of-Service (QoS) and avoid Service Level Agreement (SLA) violations at runtime is significantly more complex compared to centralized cloud scenarios. Furthermore, security and privacy concerns must be carefully addressed at every decision level, as they are crucial in an ecosystem with a broader attack surface than cloud systems [1].

Research in this area has aimed to tackle these challenges by proposing peer-to-peer (P2P) solutions for managing network dynamics arising at the edge [12]. As for workload prediction, over the last decade, a variety of techniques have been proposed within the context of cloud computing [13]. However, little effort has been put to adopt (variants of) these techniques to P2P-based edge computing. *ML-based workload forecasting using P2P (or decentralized) learning* can be extremely advantageous in a fully-decentralized edge environment [14]: this approach takes advantage of contributions from different nodes to construct a forecasting model tailored to each node. Unlike a model trained only on a node's local data, this method offers better generalization, and preserves privacy while minimizing bandwidth utilization.

Among the variants of P2P learning, *Gossip Learning* (GL) stands out as a decentralized, asynchronous approach [15]. In GL, peers (i.e., participant nodes) communicate directly to update their models [16], eliminating the need for a central system to aggregate locally-trained models, as required in other distributed learning approaches such as Federated Learning (FL) [17]. This feature makes GL particularly suitable for P2P edge environments, as it eliminates the risk of a single point of failure and leverages the existing P2P network to build the workload forecasting model.

Received 31 July 2024; revised 19 March 2025; accepted 6 May 2025. Date of publication 15 May 2025; date of current version 7 August 2025. This research was partially funded by the European Union - Next Generation EU under the Italian National Recovery and Resilience Plan (NRRP), Mission 4, Component 2, Investment 1.3, CUP E83C22004640001, partnership on "Telecommunications of the Future" (PE000000001 - program "RESTART"). This research was funded in whole or in part by the Austrian Science Fund (FWF) [10.55776/P36870; 10.55776/PAT1668223]. For open access purposes, the author has applied a CC BY public copyright license to any author accepted manuscript version arising from this submission. The associate editor coordinating the review of this article and approving it for publication was A. Sacco. (Alessandro Tundo and Federica Filippini equally contributed to this work.) (Corresponding author: Alessandro Tundo.)

Alessandro Tundo is with the Institute of Information Systems Engineering, Technische Universität Wien, 1040 Vienna, Austria (e-mail: alessandro.tundo@tuwien.ac.at).

Federica Filippini, Francesco Regonesi, Michele Ciavotta, and Marco Savi are with the Department of Informatics, Systems and Communication, University of Milano-Bicocca, 20126 Milan, Italy (e-mail: federica.filippini@unimib.it; regonesifrancesco@gmail.com; marco.savi@unimib.it; michele.ciavotta@unimib.it).

Digital Object Identifier 10.1109/TNSM.2025.3570450

Moreover, GL addresses the challenges in developing such a model for a geographically-dispersed environment with unreliable nodes by allowing asynchronous model updates. This method ensures that information eventually disseminates throughout the entire edge network [15]. In addition, by incorporating contributions from various nodes, each node's model can capture a wide range of workload patterns, achieving a balance between generalization and specificity. This balance is essential in a decentralized learning context, allowing nodes to handle out-of-distribution traffic patterns effectively. Finally, a well-generalized model can be adopted by new nodes joining the network, even if they lack historical data to develop their own models.

This paper introduces **Gossip Learning as a solution for decentralized workload forecasting in edge environments**. While our proposed method is generally applicable to a variety of edge computing scenarios, we adopt, as a motivational example, DFaaS [18], a decentralized edge environment that implements the *serverless Function-as-a-Service* (FaaS) model. In this context, we focus on forecasting the workload—a critical step towards enabling improved responsiveness, resource scaling, and distribution at the edge [3], [18], [19], [20], [21], [22].

We enhance the GL protocol to better align with the characteristics and challenges of decentralized edge computing. In particular, we propose a *performance-based approach* inspired by PENS [23] to selectively merge models from neighbors, increasing the robustness. We also propose a novel merge strategy, *Age-Weighted Overwrite*, which uses the model *age* to probabilistically decide model overwrites. In addition, we explore *FixedUpdates* and *EarlyStopping* criteria to prevent overfitting. Moreover, we implemented and publicly released a **discrete-event simulator** capable of simulating GL with various network topologies and stopping criteria, which is a feature not available in other simulators [24], [25]. An open-source prototype of the simulator is publicly accessible [26].

Finally, we conducted an **extensive empirical evaluation** to assess the applicability of GL for workload forecasting in P2P edge topologies. In particular, we considered five dimensions: (i) the performance of GL in this domain, compared to classical approaches (i.e., offline-only, centralized, and Federated Learning); (ii) the impact of GL parameters on performance; (iii) the scalability and network overhead of GL; (iv) the robustness of GL to node failures; (v) the demand for computational resources during training. Our experiments leveraged both real workload traces from FaaS applications and synthetic instances. All experimental materials are *publicly available* on Zenodo [27] and on GitHub [26].

The rest of the paper is organized as follows. Section II provides background information that introduces the Gossip Learning protocol. Section III describes our motivational scenario and the approach we propose to tackle the problem of decentralized load forecasting. Section IV provides details on the discrete-event simulator. Sections V and VI present details of the experimental setup and empirical evaluation. Section VII discusses the related work. Finally, Section VIII outlines the conclusions and possible future research directions.

II. BACKGROUND

Learning in distributed systems, where data resides on distinct nodes, presents a challenge due to network and computational scalability limitations, and privacy concerns [17].

Federated Learning (FL) [17] was initially proposed by Google as a means of training the model underlying its predictive keyboard (GBoard) while preserving user privacy. In FL, model training is distributed across multiple workers, each holding a private local dataset. A central server coordinates the training process by periodically selecting a subset of workers, elaborating the latest global model and transmitting it to them. The selected workers perform local training on their respective datasets for a predefined number of epochs and return model updates—typically in the form of weight updates or gradients—to the server. The server then aggregates them to refine the global model before the next training round.

FL is currently widely adopted for tasks outside the original scenario, including energy consumption forecasting [28], base station traffic forecasting [29] and many others [30]. Nevertheless, a single aggregation point (i.e., the main server) is still required to reconstruct the global model, which is challenging to achieve in decentralized edge computing scenarios where nodes are generally considered unreliable due to unpredictable failures (e.g., in network links) [1].

Gossip Learning is an optimization framework for the training of ML models in decentralized environments [31]. In GL, all nodes act as equal peers, each independently training a local ML model using its private dataset. Instead of relying on a central server, nodes periodically exchange model updates with randomly-chosen peers in a decentralized, peer-to-peer manner. Through these exchanges, models gradually incorporate knowledge from different nodes as propagated across the network. Unlike Federated Learning, where a single global model is aggregated, Gossip Learning results in a diverse set of models—one per node—each shaped by the specific sequence of updates it has received.

This approach is particularly well-suited for edge environments, as it fully leverages the decentralized nature of the system without requiring a central synchronization point. Additionally, GL has been shown to achieve performance on par to or even surpassing that of FL [15], while offering enhanced robustness. By eliminating a single point of failure, GL mitigates the vulnerabilities associated with centralized architectures, making it more resilient to node failures and network disruptions.

In this work, each node participating in the GL protocol must implement a defined interface that supports two primary operations, formally defined by the asynchronous procedures `MainLoop` and `OnReceiveModel`, as detailed in Algorithm 1. The `MainLoop` routine periodically disseminates model updates w along with contextual information c (e.g., the node's univocal identifier) to a subset of its peer nodes.¹ The `OnReceiveModel` procedure is triggered

¹Model updates may consist of the full set of model parameters, a subset of it, or the model's gradient computed with respect to local data. This work adopts a representation where updates refer to (a subset of) model weights, aligning with the implementation considered in this paper.

Algorithm 1 Gossip Learning Routines

```

1: async procedure MAINLOOP()
2:   while true do
3:     peers  $\leftarrow$  SELECTPEERS()
4:     SEND( $(c, w)$ , peers)
5:     wait( $\Delta_t$ )  $\triangleright \Delta_t$  is the model transmission time
6:   end while
7: end procedure

8: async procedure ONRECEIVEUPDATE( $c_r, w_r$ )
9:    $(c, w) \leftarrow$  MERGE( $(c, w)$ ,  $(c_r, w_r)$ )
10:   $(c, w) \leftarrow$  TRAIN( $(c, w)$ ,  $D_l$ )  $\triangleright D_l$  is the node dataset
11:   $c \leftarrow$  UPDATECONTEXT( $(c, w)$ ,  $(c_r, w_r)$ ,  $D_l$ )
12: end procedure

```

upon receiving an update from a peer node, integrating the received model with the local one, fine-tuning it using the node's dataset, and updating the context.²

The following Sections II-A and II-B provide further details about peer selection and merge strategies.

A. Peer Selection

Indiscriminately broadcasting model updates to a large number of neighbors is inefficient and impractical under bandwidth constraints. In GL, nodes—particularly those with high connectivity—must carefully select a subset of peers for model transmission to balance communication overhead and learning efficacy. The `selectPeers` routine plays a crucial role in managing the trade-off between exploration and exploitation in decentralized learning. It is responsible for peer discovery [32] and selecting one [15] or, rarely, more peers [33] to send the local model (unicast vs. multicast).

Peer selection strategies can be broadly classified as *static* or *dynamic*. Static strategies maintain a fixed set of target peers throughout training, ensuring stable communication patterns but potentially limiting convergence speed, reliability, and model diversity. In contrast, dynamic strategies adaptively select peers at each iteration, introducing variability that may enhance learning efficiency.

The effectiveness of a peer selection strategy depends on the underlying network topology and application requirements. In fully-decentralized settings, random peer selection provides robustness to node failures [34]. Other methods incorporate topology-aware selection and node prioritization based on factors such as model similarity, learning progress, or network proximity [35], [36], [37], improving convergence speed and reducing communication overhead. As specified in Section III-C, in this paper we adopt a strategy called *Active-Peer Random Selection*, due to its high robustness to failures.

B. Merge Strategy

In distributed/decentralized learning, the Merge strategy governs how local models are integrated, shaping both convergence dynamics and model diversity. These strategies can

be classified based on the nature of exchanged data, the aggregation function employed, and their adaptability.

Parameter-based merging aggregates model parameters through methods such as simple or weighted averaging, ensuring stability but often struggling in non-iid settings, where statistical heterogeneity degrades performance [38]. Notable examples include *Overwrite*, and *Simple Average* [15]. In *Overwrite*, the received model completely replaces the current one. In *Simple Average*, the arithmetic mean of the received model weights and the local model weights is computed to mitigate the risk of complete information loss.

Adaptive merging dynamically adjusts aggregation weights based on inter-model similarity, thereby improving generalization in heterogeneous conditions [15]. Conversely, *gradient-based merging* transmits only gradient updates—exemplified by Decentralized Stochastic Gradient Descent (DSGD)—offering communication efficiency but introducing *gradient staleness*, which may hinder optimization [39]. While research on GL often draws from FL, a direct transfer of insights remains problematic, as FL assumptions do not seamlessly extend to GL settings. A critical gap persists in the literature, underscoring the need for targeted investigations into the behaviors emerging within GL frameworks.

As specified in Section III-C, in this paper we consider, in addition to *Overwrite* and *Simple Average*, two strategies named *Age-Weighted Average*, firstly introduced in [15], and the novel *Age-Weighted Overwrite*.

III. WORKLOAD FORECASTING WITH GOSSIP LEARNING

This section presents our motivational scenario for workload forecasting in decentralized edge environments. Details about how the GL protocol was adapted to align more closely with the specific characteristics of the problem at hand are then provided and discussed.

A. Motivational Scenario

Several pertinent use cases for advanced computation at the edge of the network have been identified in the literature. These include mobile augmented reality (MAR) [16], [40], traffic safety applications [41], and IoT data processing [3], [4], [5] just to mention a few.

A reference scenario is defined herein, as illustrated in Fig. 1, which depicts a network of telecommunication towers equipped with computing and storage capabilities, thereby functioning as edge computing nodes (ENs). Each EN hosts an instance of the DFaaS platform, as initially proposed by Ciavotta et al. [18].

DFaaS (Decentralized Function-as-a-Service) is a peer-to-peer edge computing platform that implements the Function-as-a-Service paradigm, enabling each node to execute *serverless functions* in response to client requests. Clients, such as smartphones or other smart devices, initiate function execution requests by selecting the node with the strongest perceived signal—typically the nearest one. When a node's *predicted incoming workload* exceeds its available computing resources, the node dynamically offloads excess requests to

²For clarity and generality, context updates are explicitly represented by the `UpdateContext` procedure in Algorithm 1. However, they may also be handled within `MERGE` or `TRAIN`, depending on the specific implementation.

Algorithm 2 Modified OnReceiveModel for Multi-Model Merging

```

1: procedure ONRECEIVEMODEL( $c_r, w_r$ )
2:    $l_r \leftarrow \text{GETLOSS}(D_l, w_r)$   $\triangleright$  Evaluate model on local data
3:    $\text{cache.store}((c_r, w_r, l_r))$ 
4:   if  $\text{cache.size}() > T$  then:
5:      $\text{models} \leftarrow \text{cache.get\_best\_n}(N)$ 
6:      $w \leftarrow \text{MERGE}(\text{models})$ 
7:      $w \leftarrow \text{TRAIN}(w, D_l)$ 
8:      $c \leftarrow \text{UPDATECONTEXT}(c, w, D_l)$ 
9:      $\text{cache.clear}()$ 
10:  end if
11: end procedure
  
```

where network topology and workloads shift over time. To address this limitation, we propose a peer selection mechanism that remains resilient under such changes.

Rather than relying on feature similarity, we leverage the DFaaS P2P network's peer sampling service to enable decentralized and scalable neighbor discovery. It prioritizes peers based on latency, selecting topologically-closer nodes to balance similarity with reduced communication overhead. This locality-aware selection provides a crucial advantage: it inherently adapts to evolving network conditions, seamlessly integrating new nodes without requiring explicit similarity estimation. Furthermore, unlike traditional GL, which predominantly relies on unicast propagation, we employ multicast to improve communication efficiency. To prevent excessive message flooding, we introduce a stochastic selection mechanism that restricts interaction to a controllable fraction of active peers. This ensures a diverse model exchange while keeping message overhead manageable.

3) *Age-Weighted Overwrite*: In this paper, we propose a novel model merging strategy, termed the *Age-Weighted Overwrite* strategy, which synthesizes the *Overwrite* approach (discussed in Section II-B) with the *Age-Weighted Average* method, as introduced in [15].

The *Age-Weighted Average* [15] strategy modulates the merging process by incorporating a weighting factor that accounts for the *age* of each model. Formally, the age of model i , denoted as t_i , is defined as the cumulative number of data records on which the model has been trained, where each record is counted anew upon every training pass. This formulation ensures that models trained on larger or more frequently used datasets are assigned higher ages, reflecting their accumulated learning history. The primary objective of this approach is to mitigate the adverse effects of integrating *outdated* models—i.e., models trained on substantially fewer data points than the current one. The model age information can be communicated through the context c , facilitating efficient coordination among distributed agents.

The proposed *Age-Weighted Overwrite* strategy introduces a probabilistic *overwrite* mechanism, wherein the received model may entirely replace the current model with probability p , which is dynamically computed based on the relative ages (defined as in [15]) of the models. Specifically:

$$p = \begin{cases} 1 - \left(\frac{t}{t_r}\right)^2, & \text{if } t_r > t, \\ \left(\frac{t_r}{t}\right)^2, & \text{if } t_r \leq t. \end{cases}$$

with t and t_r representing the ages of the local and received models, respectively.

4) *Stopping Criteria*: In its classical form, GL assumes that the protocol is either continuously running or running for a fixed amount of time. While the literature offers numerous sophisticated techniques for terminating training to prevent overfitting (e.g., early stopping, as discussed in [44]), these methods are often challenging to adapt to the GL paradigm. This is because each node operates with a distinct model, making it difficult to define a general stopping criterion that can effectively control the entire procedure.

In this work, we propose and examine the potential of employing two stopping criteria for GL in our scenario:

- 1) *FixedUpdates*: each node is constrained to perform a fixed number of updates. A new update is performed each time a node receives a message from a neighbor. In the event that a message is received subsequent to the threshold being reached, it is discarded. This is the simplest criterion, but when applied it does not take into account the quality of the resulting model.
- 2) *EarlyStopping*: each node implements an early stopping mechanism. Upon each update, the merged model is trained for a fixed number of epochs. Subsequently, the validation loss is evaluated and compared to that of the best trained model so far. If the validation loss does not show an improvement of at least a quantity equal to the parameter designated as `min_delta`, a counter is increased; otherwise, it is reset to zero. Upon reaching a value designated as `patience`, the stopping criterion is met. The primary distinction between this approach and a classical early-stopping procedure [44] is the way how the validation loss is evaluated. In traditional approaches, the comparison is conducted between a global reference model and a local one. In this approach, however, the validation loss is evaluated after each update on a model—result of a merge operation—that may originate from a distinct neighbor with respect to previous merged models.

In both criteria, GL must address the issue of nodes terminating their training at a different point in time. This presents a challenge, as other nodes may continue selecting a node that is no longer active in training activities, resulting in the inefficient utilization of network resources and a slower convergence speed. Nevertheless, this issue can be addressed by establishing two fundamental rules:

- 1) Whenever a node receives a message including a model after its stopping criterion has been satisfied, it discards it and notifies the sender that it is no longer active;
- 2) A node must keep the `MainLoop` running for as long as it has some active neighbors, regardless of whether it has stopped or not, to ensure that its model—which is final if it has met its stopping criterion—can be sent to them. This ensures that those neighbors can continue updating their models until they also meet the stopping criterion.

More details on node states and transitions in our simulations will be provided in Section IV-A.

5) *Model Compression via Subsampling*: The exchange of model updates among peers in decentralized learning systems

introduces significant network overhead, particularly as models grow in size and complexity. This communication burden not only affects scalability but also exacerbates vanishing variance.

To address these challenges, *model compression* has been proposed as a means to reduce network overhead, mitigate vanishing variance, and enhance generalization [45], [46], [47]. A seminal work in this area explores strategies for improving communication efficiency in FL through compression techniques such as quantization, sparsification, low-rank approximation, and subsampling [45]. However, in the specific domain of GL, the use of model compression techniques remains underexplored. A handful of studies have proposed subsampling, where only a fixed percentage of randomly-selected model weights is shared with selected peers [15], [43]. Building upon this body of work, our paper investigates *subsampling* as a model compression technique due to its proven computational efficiency.

IV. DISCRETE-EVENT SIMULATOR

We designed a *discrete-event simulator* capable of running both the GL classical protocol and the proposed extensions. The simulator is scenario-agnostic, meaning it is not restricted to the specific context described in Section III-A.

Events are processed sequentially based on their timestamps, that is, when a new event occurs, it is added to a time-ordered *priority queue*, whose elements are processed by the simulator one at a time. The simulation ends when the queue is empty. Optionally, a simulation can be configured so that each node will periodically test its current model against a set of benchmark models, which shall be the same for all nodes (e.g., a model trained in a centralized fashion). By computing the loss function for the predictions made at each step, it is possible to analyze the training progress, both in terms of loss reduction and convergence speed among the models in different nodes.

The simulator allows configuring the capacity of each link by specifying the time required to transfer model weights—expressed in simulated seconds. It is possible to define both indirect and direct links to create a wide variety of network topologies, with configurable download and upload bandwidths. Finally, the simulator supports both link and node failures in a customizable way. We have integrated both the *carbontracker* [48] and the *scalene* [49] profilers to gather information on the power, energy, CPU and memory consumption in several phases of the GL protocol.

In the following, we detail the most relevant aspects of the simulator: the support of different node states and transitions between them, and the simulation workflow including the types of generated events.

A. Node States and Transitions

Nodes can be in four different states, namely, ACTIVE, TRAINING, STOPPED, and FAILED. A node is ACTIVE if it continues accepting incoming messages from neighbors and updating its own model. A node is TRAINING when an *update* is triggered. This occurs when an ACTIVE node receives a message from a neighbor and later begins training

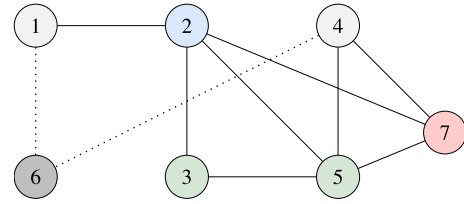


Fig. 2. Network with 7 nodes with different states. Green nodes (3 and 5) are ACTIVE, blue node (2) is TRAINING, gray nodes (1, 4, and 6) are STOPPED, and the red node is FAILED. Node 6 is not sending its model weights because its neighbors (nodes 1 and 4) are also stopped.

its merged model. Upon completion of the training, depending on whether the stopping criterion is met, the node can return to the ACTIVE state or transition to the STOPPED one. All messages received while the node is in the TRAINING state are discarded. A node is STOPPED when it meets the stopping criterion for training. Periodically, every node undergoes a binomial trial with probability p to determine if it should transition to the FAILED state. In this state, the node ceases processing and discards all incoming messages. After a configurable delay, it reverts to the ACTIVE state.

Fig. 2 shows an example network with 7 nodes. Green nodes are ACTIVE (i.e., nodes 3 and 5), node 2 in blue is TRAINING, three nodes are STOPPED (i.e., gray nodes 1, 4, and 6), and 1 node is FAILED. Among STOPPED nodes, only node 6 (dark gray) is not sending its model weights, because both its neighbors are also stopped (i.e., nodes 1 and 4).

B. Workflow and Event Types

The simulator supports five main event types, i.e., *ReceiveModel*, *SendModel*, *SaveModel*, *FailedNodeEvent*, and *RecoveryNodeEvent* that will be introduced in the following by describing the simulation workflow.

According to the protocol specification, each node must periodically send a copy of its model weights to the target neighbors. To do so, it should first check if it has any ACTIVE neighbor. If not, the event processing terminates and the node stops sending model weights. Otherwise, the node selects some targets to which it will send a copy of its serialized (and possibly compressed) model weights. Both target selection and model compression strategies depend on the current simulator configuration. After that, the simulator triggers the *ReceiveModel* events for the selected target neighbors and the next *SendModel* event for the current node, which occurs right after the transmission of all models to neighbors is completed.

Please note that during the initialization phase of a simulation, an instance of the *SendModel* event is generated for each of the nodes and enqueued in the first 60 seconds of simulated time in a random fashion. This ensures that the nodes start building their model asynchronously, eliminating the bias of a sequential start.

As said, a node can receive messages from its neighbors only if it is in ACTIVE state; otherwise, any received message is discarded. Upon receiving a message, the model weights are stored locally until T models are received from distinct

nodes. Once this threshold is reached, the N best performing ones, in addition to the local one, are merged into a single model using the chosen merge strategy, and this undergoes training for a fixed number of epochs. After the training process is completed, the `SaveModel` event is triggered. Separating these two events allows the simulator to evaluate the computational time required to conduct the training.

Upon completion of a round of model updates, a node must evaluate the new model performance by computing the validation loss. If this is lower than the one of the previous model, indicating improved performance, the node should store the updated model parameters. Additionally, the node must assess its stopping criterion to determine its next state.

Finally, at regular intervals, the simulator probabilistically determines whether a node should transition to the failing state. If so, it generates both a `FailedNodeEvent`, marking the node's transition to a state where it halts processing and discards incoming messages, and a `RecoveryNodeEvent` with a future timestamp, scheduling the node's return to the `ACTIVE` state. An open-source prototype of the simulator is publicly accessible [26].

V. EXPERIMENTAL SETUP

This section provides details on the datasets preparation (Section V-A), the ML model used for all the experiments (Section V-B), and finally, the training methodology and evaluation metrics we considered (Section V-C).

A. Datasets Preparation

We considered two different types of workload traces as datasets for our experiments: synthetic traces, which we considered to perform an initial training campaign and an ablation study validating the different components of our GL protocol, and real traces extracted from the workload of Microsoft's Azure Functions [50]. In both cases, since no information is provided in the Azure dataset about which nodes serve the incoming requests, the mapping between workload traces over time and the nodes in the P2P network has been generated based on real data related to the positions of telecommunication towers and the movement of people within the city of Porto (Portugal). The underlying assumption behind exploiting mobility traces in the data generation process is that people's movement within a city is a good proxy for network traffic generated by users at the far edge.

In the following, we focus on the adopted procedures for network and traffic generation. In particular, the first and last steps are common to both the synthetic and real-traces dataset, while the traffic generation process has been carried out differently in the two cases.

1) *Network Generation*: The first step involves generating a network of interconnected nodes distributed over a geographical area. Specifically, several networks have been generated by considering the central area of Porto city (latitude in [41.1369, 41.1690], longitude in [-8.6338, -8.5862]) and sampling real antenna positions from the OpenCellID⁴ public dataset. An example is shown in Fig. 3.

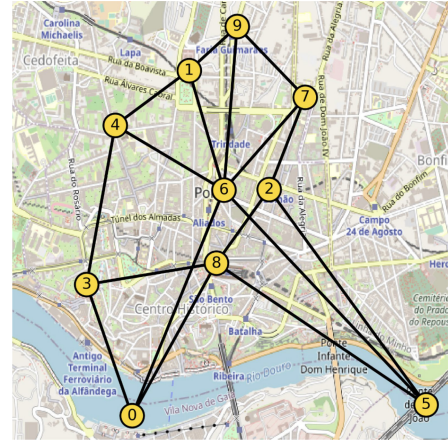


Fig. 3. Example of a generated 3-edge connected network with 10 nodes.

After selecting the desired number n of nodes, we generated networks with the desired degree of edge connectivity k ⁵; note that, considering telecommunication networks, it is safe to assume a connectivity greater than 1 to ensure a minimum level of redundancy and resilience against link failures.

Our algorithm weights the links according to *Harvesine distance* between nodes, which is a measure of the distance between two points on the surface of a sphere. This metric is often used to compute geographical distances, especially in navigation, as it provides more accurate measures than the classical Euclidean distance.

2) *Traffic Generation*: This step has been detailed differently according to whether synthetic or real function traces are considered. First of all, in both cases we have divided the considered region of the Porto city into a 20×20 grid, which simplifies the dataset generation process and enables a rapid creation of alternative datasets. Considering a specific network generated as in the previous stage, each grid is assigned to a different tower based on its distance from the tower locations. To guarantee a realistic traffic profile towards the towers, we have considered the *Porto taxi*⁶ dataset, which encompasses GPS trajectories of taxis operating in the city of Porto, sampled every 15 seconds, over the course of a year spanning from 2013 to 2014.

To generate the synthetic workload traces, for each cell of the grid we constructed a time series representing the number of taxis present at any given moment, and we assumed that a function request is generated whenever the GPS coordinates of a taxi are sampled and directed towards the tower associated to the grid the taxi is into. Considering the Azure function traces, instead, we proceeded by associating each application owner with one of the taxis in the Porto dataset. All requests coming from a specific owner (i.e., taxi) are then directed towards the tower associated with the grid where the taxi is at a given moment in time. It is relevant noting that the Azure traces include the number of per-minute function invocations over 14 consecutive days; in our work, we have considered aggregated

⁵A graph is k -edge-connected if it remains connected whenever less than k edges are removed; this occurs if each node has at least k neighbors.

⁶<https://www.kaggle.com/datasets/craita/taxi-trajectory>

⁴<https://opencellid.org>

invocations over 15-minute time intervals to limit the noise in the workload signal.

3) *Pre-Processing*: For each possible configuration (n, k) , where n represents the number of nodes and k is the network edge connectivity, 10 network variants and their corresponding node datasets are generated. For simplicity, the unique pair given by a configuration and one of its instances will be referred to as *experimental setting*. The datasets produced for each experimental setting undergo a series of pre-processing steps to prepare them for training. In particular, a total of $\nu - \tau$ time sequences is generated by each node to be used for training, where ν is the number of generated observations and $\tau = 4$ is the number of past timesteps to be used for predictions. Obtained sequences are reshaped into a training record (X_i, Y_i) , where X_i is the feature array for each time step and Y_i is a scalar value representing the true number of requests to be predicted for each timestep. The 20% of prepared sequences form the test set, while the 10% of those in the training set are kept for validation.

The resulting datasets (one per node) contain a univariate time series representing the number of requests, towards a specific serverless function, that are generated by clients connected to the base station at each specific time step. Not considering the presence of multiple serverless functions is not a limitation; indeed, as shown in [51], the best approach for FaaS workload forecasting is training a single global univariate model on samples from different time series (i.e., related to different functions) and use it to predict the incoming traffic for all functions, meaning that our approach can be readily extended to consider multiple functions. Global univariate models should be preferred over alternatives because: (i) serverless functions often exhibit similar patterns, so a model performing well on one time series is likely to do so also on others; and (ii) the multiplicity of functions is too high to train either a separate univariate model for each function or a multivariate model with such a large number of outputs.

B. Machine Learning Model

The ML model trained by each node is an LSTM (Long Short-Term Memory) neural network, widely recognized as the *de facto* standard for handling time series data due to its robustness against the vanishing gradients problem [52]. Although transformers often yield better results, their superior performance for time-series forecasting is still in question [53]; the large number of trainable parameters makes it impractical to continuously serialize and transmit them over the network.

The architecture and hyperparameters tuning were conducted manually. The final model comprises 2 stacked LSTM layers with 50 units each (using the *tanh* activation function), followed by a fully connected (FC) layer with 32 units (using the *ReLU* activation function), and a single-unit output layer. A dropout rate of 20% is applied before each FC layer as a regularization technique. The Adam optimizer [54] was chosen, with an initial learning rate of 0.001.

We used the Mean Squared Error (MSE) as a loss function, as is commonly done for regression problems. MSE is particularly suitable for this application as it penalizes larger errors

more heavily, making it effective in cases where the magnitude of the error is critical, not just its proportion to the true value.

C. Training Methodology and Evaluation Metrics

For each tested *experimental setting*, including a specific set of GL protocol parameters, we run 10 different simulations so that the same configuration and network features get tested on different network variants. Each simulation produces n models $\tilde{M}_i$ (one per node), whose performance is compared with those of two benchmark approaches: *single-node* models M_i , trained exclusively on local data, and a *centralized* model M_C , trained on combined data obtained by aggregating and shuffling the training records from all nodes. For a subset of our experiments (i.e., Sections VI-A, VI-B1, and VI-B3) we also compared our approach with Federated Learning (FL) using FedAvg [17] as aggregation strategy.

We considered the MSE our main evaluation metric to compare model performance, which is also the loss function used for training as mentioned in Section V-B. Furthermore, we computed its mean (MMSE) and standard deviation (STD) across all trained models and simulations, along with the *Relative Standard Deviation* (RSD), defined as $\frac{STD}{MMSE}$. Since it is scale-invariant, RSD provides a meaningful measure of how similar the errors obtained by different models are.

These metrics were computed in the *classical* way by comparing the predictions made by a node model with the true values contained in its local test set. However, this was not sufficient to evaluate the generalization capabilities of the trained models, whose optimization is the main goal of a decentralized learning approach (see Section I). Therefore, we defined *generalization metrics* by computing MSE, MMSE, STD and RSD, for each model, using as test set the union of all the node neighbors test sets. In the latter case, it is thus possible to evaluate the capability of the model to recognize patterns experienced by other nodes.

VI. EMPIRICAL EVALUATION

This section presents and analyzes the results of all experiments performed to validate the GL protocol in our context. In particular, a preliminary comparison among the centralized approach, the single-node (i.e., local-only) training, and GL featuring different merge strategies is conducted in Section VI-A. Section VI-B presents the results of an ablation study aimed at exploring the impact of the different GL protocol configuration parameters, and a scalability analysis is discussed in Section VI-C. A summary of our observations on how different techniques that reduce the network usage affect the GL generalization capabilities and model training times is reported in Table I. Section VI-D analyses the resilience of GL with respect to the failure of different percentages of nodes during training. A validation of our GL protocol considering real function invocation traces coming from the Microsoft Azure dataset [50] is reported and discussed in Section VI-E. Finally, GL is assessed in terms of energy and resource utilization in Section VI-F.

TABLE I
IMPACT OF DIFFERENT STRATEGIES THAT REDUCE NETWORK USAGE ON
GL GENERALIZATION CAPABILITIES AND TRAINING TIME

Technique	Impact on	
	Generalization	Training time
Reduce the number of updates	Negative	Positive
Introduce early stopping	Negative	Positive
Weights sampling	Negative	No impact
Slow down the MainLoop	No impact	Negative
Low target sampling probability	No impact	Negative

TABLE II
BASELINE NETWORK AND GL PROTOCOL CONFIGURATION

Parameter	Value
Nodes n	10
Edge-Connectivity k	3
Updates	1 epoch $\times$ 100 updates
Target sampling probability π	50%
Weights sampling	No

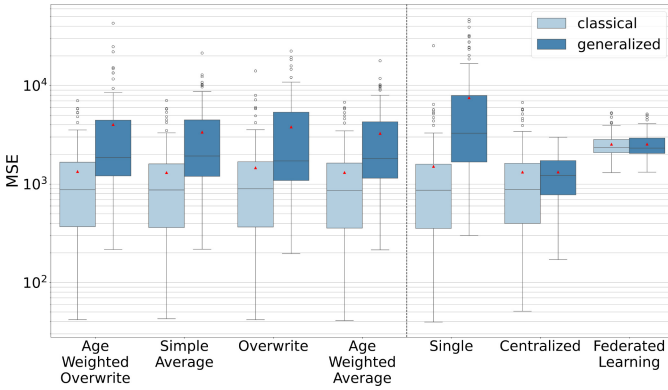


Fig. 4. Comparison among GL with different merge strategies, single-node and centralized training, and Federated Learning (red triangles denote the average MMSE across all experiments).

A. Comparison With Benchmark Approaches

Our initial experiments evaluate whether GL can serve as a viable alternative to *single-node*, *centralized*, and *Federated Learning* approaches. To do so, we conducted experiments using configuration parameters consistent with those reported in the literature [15] (see Table II), comparing the impact of different merge strategies. In these experiments, we did not consider the protocol extensions proposed in Section III-C apart from using the *Age-Weighted Overwrite* merge strategy.

We performed ten simulations across all the generated variants of a 10 nodes 3-edge connected network. The MSE and average MMSE values, considering both the *classical* and *generalized* version (see Section V-C), are reported in Fig. 4.

We observe that the values of metrics computed in the *classical* way are roughly equivalent for all approaches except for FL. In contrast, the generalization performance of GL models is better than those of *single-node* models, even though worse than the ones of the centralized model, and FL, which, however, shows a reduced variance. Notably, FL achieved generalization performance on par with GL while exhibiting reduced variance. This outcome is anticipated, as FL trains a unified model for all nodes by averaging their parameters,

TABLE III
THE MEAN AND STANDARD DEVIATION OF MSE FOR DIFFERENT
NUMBERS OF UPDATES AND TRAINING EPOCHS

Training	Classical			Generalization		
	MMSE	STD	RSD	MMSE	STD	RSD
GL 1 ep. $\times$ 100 updates	1309	1454	1.11	3256	3189	0.98
GL 1 ep. $\times$ 20 updates	2584	6049	2.34	6232	7778	1.25
GL 1 ep. $\times$ 50 updates	1404	1707	1.22	3884	3984	1.03
GL 2 ep. $\times$ 20 updates	1386	1621	1.17	5594	7034	1.26
GL 2 ep. $\times$ 50 updates	1303	1440	1.11	3822	3939	1.03
GL 5 ep. $\times$ 20 updates	1292	1425	1.10	5307	6798	1.28
GL 5 ep. $\times$ 50 updates	1270	1389	1.09	4814	5212	1.08
FL 1 ep. $\times$ 100 updates	2530	806	0.32	2535	802	0.32
FL 1 ep. $\times$ 20 updates	3189	1130	0.35	3192	1134	0.36
FL 1 ep. $\times$ 50 updates	2452	689	0.28	2453	692	0.28
FL 2 ep. $\times$ 20 updates	2575	830	0.32	2571	830	0.32
FL 2 ep. $\times$ 50 updates	2553	705	0.28	2567	724	0.28
FL 5 ep. $\times$ 20 updates	2718	932	0.34	2721	918	0.34
FL 5 ep. $\times$ 50 updates	2730	967	0.35	2728	958	0.35
Single (100 ep.)	1510	2768	1.83	7544	9852	1.31
Centralized (100 ep.)	1322	1403	1.06	1324	701	0.53

thereby slightly penalizing individual node performance in favor of enhanced overall generalization. Furthermore, the *Age-Weighted Overwrite* strategy, although marginally inferior to the *Age-Weighted Average* strategy in terms of MMSE, still slightly improves the base *Overwrite* strategy in terms of classical metrics. Finally, we can observe from the extension of box-plot bars that, with the only exception of the generalization performance of the *centralized* model, all the other strategies achieve comparable values of STD and RSD. Collectively, these results underscore the robustness of the GL protocol with respect to the adopted merge strategy across both classical and generalization metrics.

B. Impact and Robustness of GL Protocol Parameters

We experimented with various configurations of the GL protocol to evaluate how they balance model generalization capabilities, network transmission costs (i.e., network usage), and training time. In all subsequent sections, except for Section VI-B3, we present results using the *Age-Weighted Average* merge strategy. This strategy is slightly more effective in terms of generalization metrics.

1) *Number of Epochs Per Update and Number of Updates:* We evaluated the impact on performance and convergence speed of varying the number of updates per node and training epochs per update, while keeping all other parameters unchanged. The results presented in Table III indicate that increasing the number of epochs per update can yield performance comparable to or even superior to that of a higher number of updates with fewer epochs, based on classical metrics. Regarding generalization power, as expected, reducing the number of updates negatively impacts performance. However, the GL models still outperform those trained on a single node, and different configurations of FL for classical metrics, suggesting that a balance can be achieved between desired accuracy and communication frequency among nodes.

This behavior happens because, when increasing the number of epochs per update, models have more time to fit a node dataset after they are transferred. This increases the model divergence (see Fig. 5), but at the same time implies a faster and more robust training progress with respect to the

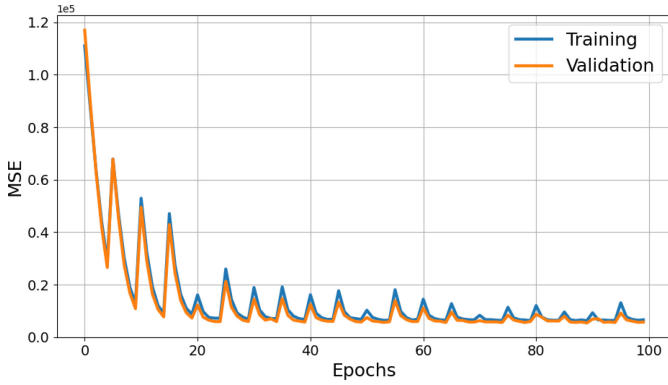


Fig. 5. MSE during training for one node during a sample experiment with 20 updates $\times$ 5 training epochs each. A spike occurs whenever a new model arrives (the most it differs from the local model, the highest).

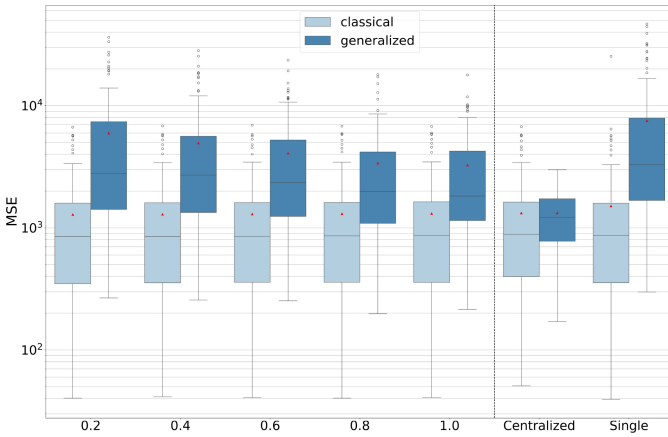


Fig. 6. Comparison between GL with different compression rates and benchmark methods (red triangles: average MMSE across all experiments).

predictions performed on the local node datasets, albeit at the expense of generalization.

2) *Weights Compression Through Subsampling*: This approach applies only to average-based merge strategies, intending to reduce the message size and, consequently, network usage (which decreases proportionally to the sampling rate if the number of updates per node remains constant).

We considered sampling rates of 0.2, 0.4, 0.6, 0.8, and 1 (i.e., no sampling), and set the other parameters as in Table II. Experiments results (see Fig. 6) highlight how higher sampling rates improve the performance in terms of classical metrics, since they limit the impact that neighbors have on the model development in each node. As before, while generalization capabilities are negatively affected by model sampling, the MMSE of GL models remains significantly lower than the one achieved by the *single-node* model, which promotes the effectiveness of the paradigm.

3) *Peer Sampling*: One simple method to lower the number of sent messages, thereby reducing network costs, is to decrease the *target sampling probability* π , i.e., the likelihood that a node will choose to communicate with its target neighbors. Table IV, experiments with π values ranging from 0.25 to 1.0 reveal that variations in π exert only a marginal

TABLE IV
MSE MEAN AND STANDARD DEVIATION WITH DIFFERENT TARGET SAMPLING PROBABILITIES π

Experiment Strategy	π	Classical			Generalization		
		MMSE	STD	RSD	MMSE	STD	RSD
GL Age-Weighted Avg.	25%	1303	1447	1.11	3304	3285	0.99
GL Age-Weighted Avg.	50%	1309	1454	1.11	3256	3189	0.98
GL Age-Weighted Avg.	75%	1314	1462	1.11	3322	3264	0.98
GL Age-Weighted Avg.	100%	1311	1449	1.11	3276	3403	1.04
FL FedAvg	25%	2835	1095	0.39	2828	1102	0.39
FL FedAvg	50%	2530	806	0.32	2535	802	0.32
FL FedAvg	75%	2728	1783	0.65	2740	1809	0.66
FL FedAvg	100%	2460	618	0.25	2444	625	0.26
Single		1510	2768	1.83	7544	9852	1.31
Centralized		1322	1403	1.06	1324	701	0.53

TABLE V
MSE MEAN AND STANDARD DEVIATION WITH DIFFERENT NUMBERS OF MERGED MODELS

N. Models	Sampling	Classical			Generalization		
		MMSE	STD	RSD	MMSE	STD	RSD
1	100%	1309	1454	1.11	3256	3189	0.98
3	100%	1312	1464	1.12	2893	2601	0.90
1	20%	1285	1417	1.10	5990	7345	1.23
3	20%	1295	1439	1.11	4704	5446	1.16

influence on the performance of GL, while FL displays higher sensitivity, affecting both *classical* and *generalization* metrics.

4) *Number of Merged Models*: To evaluate the impact of the PENS-inspired variant described in Section III-C1, we run two experiments considering $N = 3$ merged models, one with no compression and one with 20% weights sampling. In both cases, we observed an improvement in terms of generalization capabilities, while the classical metrics remain roughly unchanged (see Table V).

However, it is relevant to note that using such a technique increases the pressure on the network: since each node has to wait until it receives a message from $N > 1$ (here, 3) different neighbors before triggering an update, the overall number of the exchanged messages will be at least N times higher than in the standard case ($N = 1$). In any case, note that an $N \times$ estimate may be too optimistic: in general, nodes do not receive *exactly* one message per update from each of its neighbors; instead, multiple messages are exchanged before collecting the N required sets of model weights.

5) *Stopping Criteria*: We run a set of experiments to investigate the impact of the two stopping criteria introduced in Section III-C4: *FixedUpdates* and *EarlyStopping* (ES). With respect to the latter we want to investigate whether, from a node perspective, its own validation loss history may be a valid indicator of the overall training progress. In particular, we designed a *conservative EarlyStopping* strategy that requires 5 updates without an improvement of at least 0.1 in validation loss, and an *aggressive EarlyStopping* one, which requires 5 epochs without an improvement of at least 1. We compare it with a *FixedUpdate* strategy where training is performed with 100 fixed updates per node.

As shown in Table VI, even though the *classical* metrics are roughly comparable, models trained with *EarlyStopping* show remarkably worse generalization performance than those trained for more epochs. Based on these results, it is reasonable to assume that validation loss is not a sufficient metric to define

TABLE VI
MSE MEAN AND STD DEVIATION WITH DIFFERENT STOPPING CRITERIA

Schedule	Classical			Generalization		
	MMSE	STD	RSD	MMSE	STD	RSD
<i>FixedUpdates</i>	1309	1454	1.11	3256	3189	0.98
<i>Conservative ES</i>	1328	1451	1.09	6267	8389	1.34
<i>Aggressive ES</i>	1334	1461	1.09	6392	8082	1.26

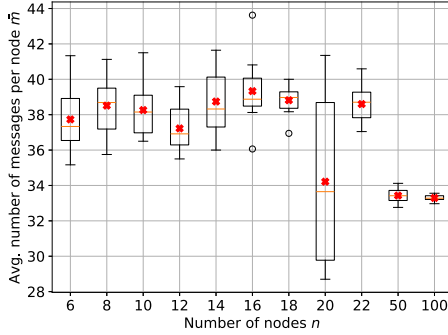


Fig. 7. Average number of messages per node with respect to the number of nodes in the network. Each blue data point represents the $\bar{m}$ value within a single simulation; red crosses represent the average over all the simulations.

an intelligent stopping criterion as it does not take into account the generalization performance of models.

At the same time, we can observe that the generalization capabilities of GL models are still superior to those of a single-node-only model. Since *EarlyStopping*, limiting the number of updates per node helps reduce the network usage and the training time, a good trade-off can be found between the models' accuracy and the overall costs of their development.

C. Scalability and Network Costs

Scalability is one of the most relevant properties for distributed approaches. We analyzed how the GL protocol scales to bigger P2P networks by considering as metric the *average number of messages exchanged per node* throughout the simulation, denoted by $\bar{m}$. In particular, Sections VI-C1–VI-C4 evaluate whether and how this is influenced by the protocol configuration parameters. Section VI-C5 discusses whether GL can limit, with respect to the *centralized* approach, the network load each node is subject to. All experiments except those in Section VI-C4 were run considering 25 updates per node, and assuming that 4 epochs are trained per update.

1) *Impact of the Number of Nodes in the Network*: The first set of experiments explored the relationship between $\bar{m}$ and the number of nodes n , varied between 6 and 22, and two larger networks with 50 and 100 nodes. All experiments were conducted on 3-edge connected P2P networks. As expected, $\bar{m}$ is roughly constant with respect to n (see Fig. 7). However, it is relevant to note that the *observed* value of $\bar{m}$ is always significantly higher than the theoretical minimum (which is 25, as the number of updates per node). This discrepancy arises because nodes typically exchange multiple messages before the communication process is effectively concluded.

Another key factor affecting $\bar{m}$ is the number of incoming messages a node discards, either because it is in the

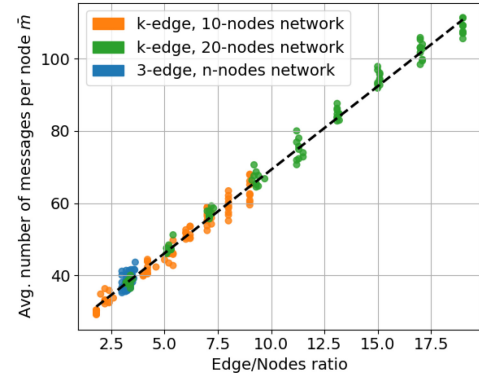


Fig. 8. Average number of messages per node for different network graphs.

TRAINING state or because it has already begun the transition to the STOPPED state (see Section IV-A). In particular, the number of messages discarded during the transition to STOPPED is relatively small, and its impact diminishes, as the number of per-node updates increases. *In contrast, the number of messages discarded during the TRAINING state is influenced by the average number of neighbors of each node (i.e., by the k -edge connectivity of the network), and by the ratio between the training and the model transfer times, as clarified in the next sections.*

2) *Impact of the Number of P2P Network Links*: Since, as mentioned, the number of discarded messages is influenced by the average degree of a network node, it is interesting to see how network costs are impacted by an increase in the number of network links. For this reason, three sets of experiments were performed. We initially considered 10-nodes and 20-nodes networks, respectively, and tuned the k parameter related to the k -edge connectivity by considering: (i) all values between 1 and 9 for the 10-nodes network; (ii) all odd values between 3 and 19 for the 20-nodes network. Then, we kept $k = 3$ fixed and varied n between 5 and 20 as in the previous section.

Note that, as mentioned in Section V-A1, k -edge connectivity is guaranteed if each node has *at least* k neighbors. However, the actual number of edges may be slightly different depending on the generated topology. Therefore, to better characterize the network graphs, we will report in the next figures their *edge/nodes ratio* instead of the prescribed k value.

Figure 8 reveals a linear relationship between the ratio of edges to nodes (i.e., the average number of links per node) and $\bar{m}$ across all experimental settings. Moreover, the uniformity of the linear regression coefficients, combined with the analysis in the previous section, leads us to conclude that $\bar{m}$ is independent of the number of nodes in the network and depends solely on the node degree.

3) *Impact of the Transfer Time and Target Sampling Probability*: We validated the observation that high values of $\bar{m}$ are related to a large number of message rejections by increasing by 10 times the model transfer time (i.e., *High transfer time*), under the assumption that, if this is significantly longer than the training time, the chance of receiving a message while in the TRAINING state is lower. Furthermore, since we observe fewer rejections when decreasing the target

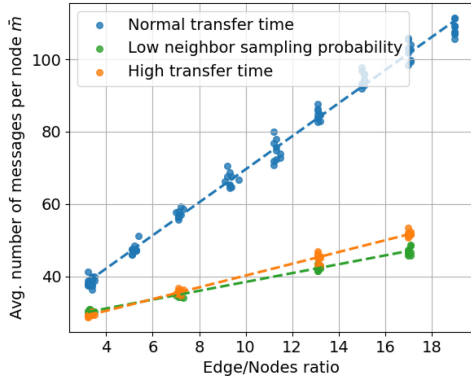


Fig. 9. Average number of messages per node for different transfer times and neighbor sampling probabilities.

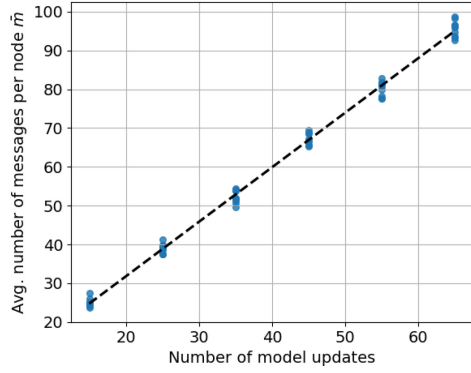


Fig. 10. Average number of messages per node varying the number of updates.

sampling probability π , we also run a set of simulations with $\pi = 0.25$ instead of 1 (i.e., *Low neighbor sampling probability*). We considered 20-nodes networks and varied k as done previously.

Results are shown in Figure 9: we observe that the values of $\bar{m}$ decrease more significantly for more dense networks. In this setting, reducing the sample probability π proves to be very effective in decreasing the network load.

4) *Impact of the Number of Model Updates*: It is reasonable to assume that $\bar{m}$ grows proportionally to the number of per-node updates within a simulation. To validate this, we run a set of experiments with a variable number of updates on a 20-node 3-edge connected network (see Fig. 10). The results of Fig. 10 confirm our assumption.

5) *Comparison With the Centralized Approach*: We compared the traffic generated by the GL protocol and the *centralized* approach considering a *star* network architecture, where the master node (i.e., centralized server) is directly linked with all the participants. In terms of network costs, this can be considered the best-case scenario for centralized training, as all the data transmissions between master and clients have only to traverse the single link connecting them.

In a centralized scenario, the incoming network cost for a master node connected to n clients is nD , where D is the dataset dimension, whereas the outgoing network traffic is nM , where M is the model size. Therefore, the overall network cost

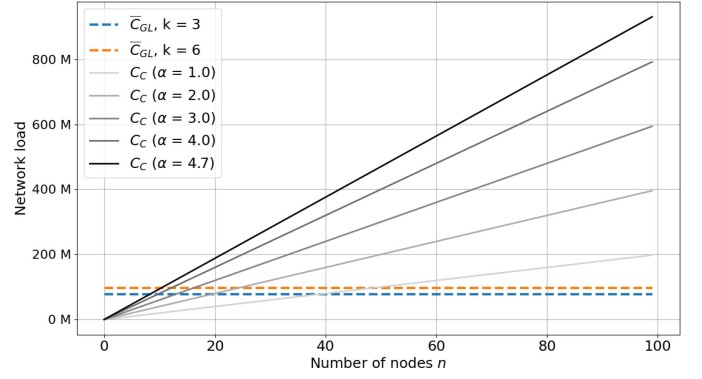


Fig. 11. Comparison between the network load with GL and the centralized approach, expressed as a function of the model size M .

can be defined as:

$$\begin{aligned} C_C &= C_C^{in} + C_C^{out} = nD + nM = n(\alpha M + M) \\ &= n(1 + \alpha)M, \end{aligned}$$

where α is a scaling factor between model and dataset size (in our experiments, α is ~ 4.7).

On the other hand, the average network load for a node participating in GL is given by:

$$\bar{C}_{GL} = \bar{C}_{GL}^{in} + \bar{C}_{GL}^{out} = r\bar{m}M + r\bar{m}M = 2r\bar{m}M,$$

where $r \in (0, 1]$ denotes the compression rate if weights sampling is applied.

As discussed in Section VI-C1, $\bar{m}$ and, consequently, $\bar{C}_{GL}$, remain invariant with respect to the number of nodes n . This invariance implies that, irrespective of the GL protocol parameters and associated network costs, there always exists a sufficiently large n for which $\bar{C}_{GL} < C_C$. An example is reported in Fig. 11, where we report the network load computed for: (i) a 3-edge connected network (with $r = 1$), (ii) a 6-edge connected network (with $r = 1$), and (iii) a centralized master node varying the value of α .

For both values of k and all α , we observe that C_C becomes higher than $\bar{C}_{GL}$ as the number of nodes reaches a given threshold. We can conclude that the GL protocol offers advantages over the centralized model in terms of network load, particularly when dealing with large graphs. While the GL protocol may generate more traffic in smaller networks, it distributes communications more evenly across the network. This reduces the likelihood of bottlenecks, which are more common when all data must be sent from clients to a centralized server.

D. Node Failure Resiliency

We assessed the resiliency of our approach to node failures by varying the failure probability. In each simulation, at fixed intervals of $i = 60$, the simulator probabilistically determines whether to trigger a node failure based on the configured probability $p \in \{0.0025, 0.005, 0.01\}$. When a failure occurs, the recovery time is sampled from a normal distribution with mean $\mu = 125$ and standard deviation $\sigma = 5$.

The parameters i , μ , and σ were chosen by considering the network size (10 nodes), the overall simulated training

TABLE VII

MEAN MSE, STD, AND RSD FOR THE CLASSICAL METRIC FOR ALL THE NODES IN THE NETWORK, NON-FAILED NODES, AND FAILED NODES UNDER DIFFERENT NODE FAILURE PROBABILITIES

Failure prob.	All nodes			Non-failed nodes			Failed nodes		
	MMSE	STD	RSD	MMSE	STD	RSD	MMSE	STD	RSD
0.0025	1306	90	0.07	1277	206	0.16	1593	1876	1.18
0.005	1307	87	0.07	1265	314	0.25	1169	646	0.55
0.01	1312	90	0.07	1523	554	0.36	1196	268	0.22

TABLE VIII

MEAN MSE, STD, AND RSD FOR THE GENERALIZATION METRIC FOR ALL THE NODES IN THE NETWORK, NON-FAILED NODES, AND FAILED NODES UNDER DIFFERENT NODE FAILURE PROBABILITIES

Failure prob.	All nodes			Non-failed nodes			Failed nodes		
	MMSE	STD	RSD	MMSE	STD	RSD	MMSE	STD	RSD
0.0025	2958	962	0.33	2813	812	0.29	5108	4835	0.95
0.005	3150	1074	0.34	3072	1049	0.34	3332	2127	0.64
0.01	3227	1170	0.36	3019	1538	0.51	3580	1446	0.40

duration at each round (5 time units), and the model weights transmission time (i.e., $t_w \in [25, 35]$). For instance, with a failure probability of $p = 0.01$, the simulation produces an average of 4 node failures, with each failure causing a node to miss approximately 3–4 training rounds before recovering.

In our experiments, runs with node failures occurred as follows: for $p = 0.0025$, 6 out of 10 runs experienced failures (10 failures in total, averaging 1.0 failure per run); for $p = 0.005$, 9 out of 10 runs were affected (18 failures in total, averaging 1.8 failures per run); and for $p = 0.01$, all 10 runs exhibited failures (53 failures in total, averaging 5.3 failures per run).

Tables VII and VIII summarize the performance results using the *classical* and *generalization* metrics, respectively. For each metric, we report aggregated values of the MMSE, STD, and RSD computed across all nodes, non-failed nodes, and failed nodes.

The results are consistent with our initial comparisons with benchmark approaches (see Fig. 4), as there is no significant performance difference between failed and non-failed nodes—demonstrating the notable resilience of our approach. An exception was observed for $p = 0.0025$, where the failed nodes exhibited an unexpected increase in MMSE (and a high STD) for the *generalization* metric. Further investigation revealed that, in one run, the sole failed node produced a substandard model (with a *generalization* MSE of 14299), which skewed the aggregated metrics. Additionally, one of its three neighboring nodes also concluded the simulation with poor performance (a *generalization* MSE of 9829), while the remaining neighbors performed comparably to the non-failed nodes. We hypothesize that this deviation is attributable to the interplay between network topology and random peer selection; however, further experiments are required to validate this hypothesis.

E. Validation With Real Function Traces

Following the same approach described in Section V-C, we trained GL models on the datasets built upon Microsoft

TABLE IX

MSE MEAN AND STANDARD DEVIATION FOR GL, SINGLE-NODE AND CENTRALIZED MODELS TRAINED AND TESTED ON THE AZURE FUNCTIONS DATASET

Training	Classical			Generalization		
	MMSE	STD	RSD	MMSE	STD	RSD
GL	27188	3348	0.12	37132	8693	0.23
Single	26843	2229	0.08	40572	9512	0.23
Centralized	24038	1991	0.08	25212	958	0.04

Azure functions traces [50] as outlined in Section V-A. We considered ten randomly-generated 10-node, 3-edge-connected networks and adopted the configuration parameters described in Table II, and the Age-Weighted Average merge strategy.

The results achieved by GL, single-node, and centralized models, both in terms of classical and generalized metrics, are reported in Table IX. The values of *classical* and *generalized* metrics confirm the pattern observed with synthetic traces, with the GL protocol improving the generalization capabilities of the models with respect to the single-node case.

Additionally, Figure 12 reports the real and predicted workload values for a sample experiment, obtained by one node participating in the GL protocol. The predictions align with both the training and test traces, as well as with the *common test trace* used to evaluate the generalized metrics. Although peak values are not correctly identified by the model in most cases, the predictions capture well the moving average of the distributions, which is represented in the figure by the green line, and closely follow the workload pattern. The same behavior can be observed in Figure 13, which reports the real and predicted traces for the common test set obtained with the single-node model and the centralized model.

F. Energy and Resource Profiling

We leveraged *carbontracker* [48] to monitor the power (in Watt) and energy (in Joule) required by specific node operations throughout the GL simulation. In particular, we focused on the model training, collecting metrics for each epoch, and on the `SendModel` and `ReceiveModel` events (see Section IV-B), gathering the power and energy consumption related to each event processing. We run simulations considering 30 updates per node, and running 10 training epochs per update. We chose to increase the number of epochs per update with respect to the experiments reported in Section VI-E to ensure that the time spent by nodes in the training and model update phases is large enough to guarantee stable measurements. We trained the models considering the dataset based on Azure functions traces, running on a MacBook Pro M3 with 11 cores and 18Gb RAM.

The average, standard deviation and quantiles of the collected metrics per each model fit, send model and receive model call are reported in Table X. As in the previous sections, they were obtained by considering 10 randomly-generated networks. Throughout the overall simulation, which corresponds to 300 training epochs and around 30 communication rounds, each node consumes (on average) 2906.29 W for model training, 310.30 W to send the model to peers, and



Fig. 12. Real and predicted workload values for a sample node participating in the GL protocol, considering the training, validation and test trace, and the common test trace used to evaluate the generalized metrics. The green line represents a moving average of the real values over 4-step windows.

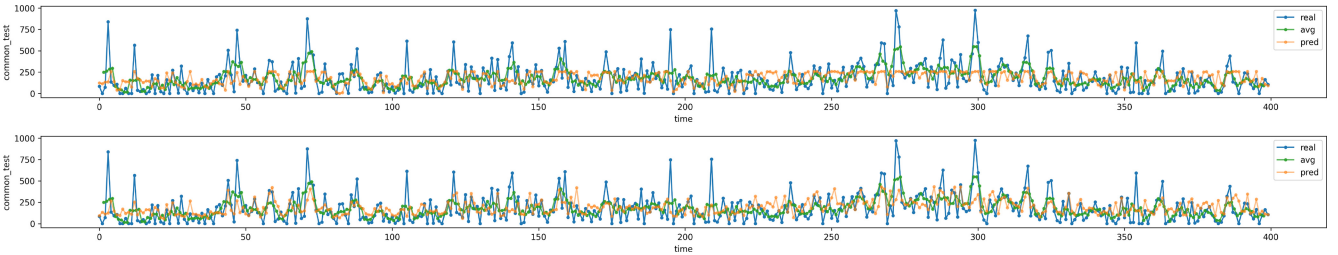


Fig. 13. Real and predicted workload values for the single-node (above) and the centralized model (below), considering the common test trace used to evaluate the generalized metrics. The green line represents a moving average of the real values over 4-step windows.

TABLE X
ENERGY CONSUMPTION IN DIFFERENT PHASES OF THE GL PROTOCOL

Event	Model fit				SendModel				ReceiveModel			
	mean	std	25%	75%	mean	std	25%	75%	mean	std	25%	75%
Power (W)	9.69	0.52	9.37	9.92	10.70	2.45	10.92	11.88	10.69	0.45	10.48	10.96
Energy (J)	1.93	3.68	0.67	0.75	0.16	0.07	0.14	0.19	20.74	1.55	19.88	21.69
Runtime (s)	0.19	0.34	0.07	0.08	0.01	0.01	0.01	0.02	1.93	0.12	1.86	1.97

320.75 W in the receive model routine. These can be used to estimate the overall CO_2 emissions by considering the average carbon intensity where simulations were conducted (in our case, Monza, Lombardy, IT, had an average intensity of $330.72gCO_2/kWh$).

Throughout the simulations, we have also gathered information on the CPU and RAM consumption, leveraging the *scalene* [49] profiler. In particular, we have observed that nodes spend most of their time in conducting the model training phase (almost 90% of the overall runtime). This is characterized by an average CPU utilization of 79.48% (with a standard deviation of 13.70) and consumes on average 27.82Mb of RAM (with a standard deviation of 0.85); the peak RAM consumption is of 41.45Mb on average (standard deviation: 3.78), which guarantees that the training process is tractable even for small-scale edge devices.

VII. RELATED WORK

This section recalls the main related work in the context of time series forecasting and edge workload forecasting, with main focus on decentralized learning.

A. Time-Series Analysis and Prediction

The field of edge workload forecasting is based on the principles of time-series analysis and prediction, a subject that has been extensively researched and documented in the academic literature for a considerable period of time [55]. In general, classical approaches (e.g., ARIMA [56]) for time-series prediction aim to forecast a single time series by studying linear relationships between observations collected in the past and future observations. Typically, this linear approach yields suboptimal predictions [55]. In recent times, there has been a notable increase in the use of neural network models, largely due to their capacity to discern non-linear relationships. For example, Recurrent Neural Network (RNN) models, such as LSTM [52] and Gated Recurrent Unit (GRU) [57], are widely used. However, these ML models often require greater time and hardware resources for training purposes [55]. In this paper we adopted LSTM given the good performance obtained in some preliminary experiments.

B. ML-Based Workload Forecasting at the Edge

Recently, research efforts have been made on the use of ML techniques specifically for workload forecasting in edge environments. For instance, Miao et al. [58] present an approach based on Graph Neural Networks to capture the interconnected topology of edge servers, assuming that servers in close proximity often exhibit similar workload patterns. Ma et al. [55] propose an edge-cloud collaboration as a means of capturing inter-site correlations while simultaneously advancing a distributed approach wherein the majority of the

computation is conducted at the Edge. Some recent work has also adopted the Federated Learning paradigm as a means to collaboratively train a model for workload forecasting at the Edge without sharing local data. For instance, Que and Khan [59] focus on this aspect in the context of Metro Optical Networks. However, these studies still either perform model training in a single location or need a central coordination, as in the case of Federated Learning. Our approach completely eliminates the need for a central location and fully decentralizes model training at the Edge.

C. Decentralized Training Solutions

When dealing with the need for decentralized training, some studies investigate the possibility of adapting FL to a decentralized setting [16], [60], [61]. Wink and Nocht [61] propose a P2P variant of FL that can enhance data confidentiality and privacy protection, particularly in systems that cater to smaller groups of collaborating data owners. In the proposed approach, peers do not exchange model weights; rather, they collaboratively perform Secure Average Computation to obtain the merged models. Kalra et al. [60] propose ProxyFL, a proxy-based FL method for decentralized collaboration that facilitates the training of robust and high-performance models while maintaining data privacy and communication efficiency. Zhou et al. [16] present PPAFL, a P2P-based privacy-perceiving asynchronous FL framework for the decentralized training of secure and resilient mobile robotic systems. These works built on top of a seminal paper by Hegedüs et al. [15], which proposes Gossip Learning as a promising decentralized alternative to FL [43]. Starting from this body of research, our work aims at adopting decentralized learning [14], with particular focus on GL, in the context of workload forecasting at the Edge, while incorporating tailored adaptations to this use case.

D. Gossip Learning Adaptations

Our proposed extensions and adaptations to GL are related to four main aspects: (i) how the received models are merged, (ii) how the peers are selected for sending model updates, (iii) how the model can be compressed to reduce communication overhead and (iv) when the protocol should stop its execution.

In literature, some different merging strategies have been proposed, being either parameter-based [15], [38], or gradient-based [39]. In our paper we adopt the merging strategies designed in [15], and we propose a novel strategy, i.e., *Age-Weighted Overwrite*, that incentivize more stable models. In addition, inspired by [23], we enable the simultaneous merging of multiple models. Different peer selection strategies have also been proposed, based, e.g., on topology-aware selection and node prioritization [35], [36], [37]. In this paper we instead adopt an *active-peer random selection* strategy, which better suits to dynamic network conditions as those occurring in the considered scenario. Model compression has been investigated in some papers [15], [43], [47]: building upon them, we propose *subsampling* as a mean to achieve model compression. Finally, no previous work on GL focuses on the problem of protocol execution's stopping to prevent overfitting. In this paper we propose two strategies, i.e.,

FixedUpdates and *EarlyStopping*, with the latter adapted from [44].

E. Time Series Forecasting With Gossip Learning

The application of GL to the forecasting of time-series data has been investigated by few works in literature. Dinati et al. [62] focus on vehicle trajectory prediction. Their scenario is similar to the one considered in this work, as it accounts for dynamic network topologies. However, our study introduces the extensions reported in Section VII-D and extends the evaluation to aspects related to scalability and network costs. Palmieri et al. [63] investigate the impact of different network topologies on the capacity of nodes to incorporate knowledge derived from data patterns observed in other nodes into their local models. Their work provides a valuable complement to our experiments on the impact of network size, focusing on the impact of network topology variation on GL.

VIII. CONCLUSION

This paper addressed the workload forecasting problem in a fully-decentralized edge environment, leveraging the Gossip Learning paradigm to mitigate challenges arising from geographical distribution, workload pattern heterogeneity, and the dynamic nature of nodes, which join and leave the network due to unstable links and failures. Building on existing peer-to-peer learning protocols, we enhanced the foundational Gossip Learning algorithm by introducing a performance-driven selection mechanism that prioritizes models received from neighboring nodes based on their quality. Additionally, we proposed a novel merging strategy and integrated two distinct stopping criteria into the protocol to mitigate overfitting. Extensive experimental validation, conducted using the discrete-event simulator developed in this work, demonstrated the effectiveness of Gossip Learning and its extensions in a representative P2P edge environment. Our results show that Gossip Learning not only achieves competitive performance against benchmark methods (i.e., centralized, single-node learning, and Federated Learning) but also exhibits robustness and scalability across diverse protocol and network conditions.

REFERENCES

- [1] X. Kong, Y. Wu, H. Wang, and F. Xia, "Edge computing for Internet of Everything: A survey," *IEEE Internet Things J.*, vol. 9, no. 23, pp. 23472–23485, Dec. 2022.
- [2] C. N. L. Tan, C. Klein, and E. Elmroth, "Multivariate LSTM-based location-aware workload prediction for edge data centers," in *Proc. IEEE/ACM CCGRID*, 2019, pp. 341–350.
- [3] C. Cicconetti, M. Conti, and A. Passarella, "A Decentralized framework for serverless edge computing in the Internet of Things," *IEEE Trans. Netw. Serv. Manag.*, vol. 18, no. 2, pp. 2166–2180, Jun. 2021.
- [4] I. Wang, E. Liri, and K. K. Ramakrishnan, "Supporting IoT applications with serverless edge clouds," in *Proc. IEEE CloudNet*, 2020, pp. 1–4.
- [5] M. S. Aslanpour et al., "Serverless edge computing: Vision and challenges," in *Proc. ACSW*, 2021, pp. 1–10.
- [6] E. Paraskevoulakou and D. Kyriazis, "Leveraging the serverless paradigm for realizing machine learning pipelines across the edge-cloud continuum," in *Proc. ICIN*, 2021.
- [7] J. Jiang et al., "Towards demystifying Serverless machine learning training," in *Proc. SIGMOD*, 2021, pp. 110–117.
- [8] P. G. Sarroca and M. S. Artigas, "MLLess: Achieving cost efficiency in serverless machine learning training," *J. Parallel Distrib. Comput.*, vol. 183, Jan. 2024, Art. no. 104764.

- [9] P. Aditya et al., "Will serverless computing revolutionize NFV?" *Proc. IEEE*, vol. 107, no. 4, pp. 667–678, Apr. 2019.
- [10] M. Savi, A. Banfi, A. Tundo, and M. Ciavotta, "Serverless computing for NFV: Is it worth it? A performance comparison analysis," in *Proc. IEEE PerCom Workshops*, 2022, pp. 680–685.
- [11] A. Sabbioni, A. Garbugli, L. Foschini, A. Corradi, and P. Bellavista, "Serverless computing for QoS-effective NFV in the cloud edge," *IEEE Commun. Mag.*, vol. 62, no. 4, pp. 40–46, Apr. 2024.
- [12] V. Karagiannis, A. Venito, R. Coelho, M. Borkowski, and G. Fohler, "Edge computing with peer to peer interactions: Use cases and impact," in *Proc. IoT-Fog*, 2019, pp. 46–50.
- [13] M. Masdari and A. Khoshnevis, "A survey and classification of the workload forecasting methods in cloud computing," *Clust. Comput.*, vol. 23, no. 4, pp. 2399–2424, 2020.
- [14] L. Yuan, Z. Wang, L. Sun, P. S. Yu, and C. G. Brinton, "Decentralized federated learning: A survey and perspective," *IEEE Internet Things J.*, vol. 11, no. 21, pp. 34617–34638, Nov. 2024.
- [15] I. Hegedüs, G. Danner, and M. Jelasity, "Gossip learning as a decentralized alternative to federated learning," in *Proc. DAIS*, 2019, pp. 74–90.
- [16] X. Zhou et al., "Decentralized P2P federated learning for privacy-preserving and resilient mobile robotic systems," *IEEE Wireless Commun.*, vol. 30, no. 2, pp. 82–89, Apr. 2023.
- [17] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. AISTATS*, 2017, pp. 1273–1282.
- [18] M. Ciavotta, D. Motterlini, M. Savi, and A. Tundo, "DFaaS: Decentralized function-as-a-service for federated edge computing," in *Proc. IEEE CloudNet*, 2021, pp. 1–4.
- [19] C. Cicconetti, M. Conti, A. Passarella, and D. Sabella, "Toward distributed computing environments with serverless solutions in edge systems," *IEEE Commun. Mag.*, vol. 58, no. 3, pp. 40–46, Mar. 2020.
- [20] C. Cicconetti, M. Conti, and A. Passarella, "Architecture and performance evaluation of distributed computation offloading in edge computing," *Simul. Model. Pract. Theory*, vol. 101, May 2020, Art. no. 102007.
- [21] T. Pfandzelter and D. Bermbach, "tinyFaaS: A lightweight FaaS platform for edge environments," in *Proc. ICFC*, 2020, pp. 17–24.
- [22] I. Baldini et al., *Serverless Computing: Current Trends and Open Problems*. Singapore: Springer, 2017.
- [23] N. Onoszko, G. Karlsson, O. Mogren, and E. L. Zec, "Decentralized federated learning of deep neural networks on non-IID data," 2021, *arXiv:2107.08517*.
- [24] R. Ormándi, I. Hegedüs, and M. Jelasity, "Asynchronous peer-to-peer data mining with stochastic gradient descent," in *Proc. Eur. Conf. Parallel Process.*, 2011, pp. 528–540.
- [25] M. Polato, "gossipy documentation," 2024. Accessed: Jul. 27, 2024. [Online]. Available: <https://makgyver.github.io/gossipy/index.html>
- [26] A. Tundo, F. Filippini, F. Regonesi, M. Ciavotta, and M. Savi, "unimib-datAI gl-forecasting-edge," 2024. Accessed: May 13, 2025. [Online]. Available: <https://github.com/unimib-datAI/gl-forecasting-edge>
- [27] A. Tundo, F. Filippini, F. Regonesi, M. Ciavotta, and M. Savi, "Zenodo repository," 2024. Accessed: May 13, 2025. [Online]. Available: <https://doi.org/10.5281/zenodo.15393791>
- [28] M. Savi and F. Olivadese, "Short-term energy consumption forecasting at the edge: A federated learning approach," *IEEE Access*, vol. 9, pp. 95949–95969, 2021.
- [29] V. Perifanis, N. Pavlidis, R.-A. Koutsiamanis, and P. S. Efraimidis, "Federated learning for 5G base station traffic forecasting," *Comput. Netw.*, vol. 235, Nov. 2023, Art. no. 109950.
- [30] H. G. Abreha, M. Hayajneh, and M. A. Serhani, "Federated learning in edge computing: A systematic survey," *Sensors*, vol. 22, no. 2, p. 450, 2022.
- [31] R. Ormándi, I. Hegedüs, and M. Jelasity, "Gossip learning with linear models on fully distributed data," *Concurr. Comput., Pract. Exp.*, vol. 25, no. 4, pp. 556–571, 2013.
- [32] M. Jelasity, S. Voulgaris, R. Guerraoui, A.-M. Kermarrec, and M. van Steen, "Gossip-based peer sampling," *ACM Trans. Comput. Syst.*, vol. 25, no. 3, pp. 8–es, 2007.
- [33] M. Assran, J. Romoff, N. Ballas, J. Pineau, and M. Rabbat, "Gossip-based actor-learner architectures for deep reinforcement learning," in *Proc. 33rd Conf. Neural Inf. Process. Syst.*, vol. 32, 2019, pp. 1–11.
- [34] D. Kempe, J. Kleinberg, and A. Demers, "Spatial gossip and resource location protocols," *J. ACM*, vol. 51, no. 6, pp. 943–967, 2004.
- [35] J. Zhang, L. Zhao, and N. Lin, "Similarity-based gossip learning for generative adversarial networks," in *Proc. ICCSN*, 2023, pp. 465–469.
- [36] L. Giarretta and Š. Girdzijauskas, "Gossip learning: Off the beaten path," in *Proc. IEEE Big Data*, 2019, pp. 1117–1124.
- [37] J. Daily, A. Vishnu, C. Siegel, T. Warfel, and V. Amatya, "Gossipgrad: Scalable deep learning using gossip communication based asynchronous gradient descent," 2018, *arXiv:1803.05880*.
- [38] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proc. MLSys*, 2020, pp. 429–450.
- [39] X. Lian, W. Zhang, C. Zhang, and J. Liu, "Asynchronous decentralized parallel stochastic gradient descent," in *Proc. ICML*, 2018, pp. 3043–3052.
- [40] Y. Wang, T. Yu, and K. Sakaguchi, "Context-based MEC platform for augmented-reality services in 5G networks," in *Proc. IEEE VTC*, 2021, pp. 1–5.
- [41] I. Lujic, V. D. Maio, K. Pollhammer, I. Bodrozic, J. Lasic, and I. Brandic, "Increasing traffic safety with real-time edge analytics and 5G," in *Proc. EdgeSys*, 2021, pp. 19–24.
- [42] Y. Gou, S. Weng, M. A. Imran, and L. Zhang, "Voting consensus-based decentralized federated learning," *IEEE Internet Things J.*, vol. 11, no. 9, pp. 16267–16278, May 2024.
- [43] I. Hegedüs, G. Danner, and M. Jelasity, "Decentralized learning works: An empirical comparison of gossip learning and federated learning," *J. Parallel Distrib. Comput.*, vol. 148, pp. 109–124, Feb. 2021.
- [44] T. Zhang, T. Zhu, K. Gao, W. Zhou, and P. S. Yu, "Balancing learning model privacy, fairness, and accuracy with early stopping criteria," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 9, pp. 5557–5569, Sep. 2023.
- [45] H. B. McMahan et al., "Federated learning: Strategies for improving communication efficiency," in *Proc. NeurIPS*, 2016, pp. 1–10.
- [46] A. Reisizadeh, A. Mokhtari, H. Hassani, A. Jadbabaie, and R. Pedarsani, "Fedpaq: A communication-efficient federated learning method with periodic averaging and quantization," in *Proc. AISTATS*, 2020, pp. 2021–2031.
- [47] H. Wang, S. Guo, Z. Qu, R. Li, and Z. Liu, "Error-compensated sparsification for communication-efficient Decentralized training in edge environment," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 1, pp. 14–25, Jan. 2022.
- [48] L. F. W. Anthony, B. Kanding, and R. Selvan, "Carbontracker: Tracking and predicting the carbon footprint of training deep learning models," in *Proc. ICML Workshop Challenges Deploying Monit. Mach. Learn. Syst.*, 2020, pp. 1–11.
- [49] E. D. Berger, S. Stern, and J. A. Pizzorno, "Triangulating python performance issues with scalene," in *Proc. 17th USENIX Symp. Oper. Syst. Design Implement. (OSDI)*, Boston, MA, USA, Jul. 2023, pp. 51–64. [Online]. Available: <https://www.usenix.org/conference/osdi23/presentation/berger>
- [50] M. Shahrad et al., "Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider," in *Proc. USENIX ATC*, 2020, pp. 205–218.
- [51] A. Joosen et al., "How does it function? Characterizing long-term trends in production serverless workloads," in *Proc. SoCC*, 2023, pp. 443–458.
- [52] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [53] A. Zeng, M. Chen, L. Zhang, and Q. Xu, "Are transformers effective for time series forecasting?" in *Proc. AAAI*, 2023, pp. 11121–11128.
- [54] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014, *arXiv:1412.6980*.
- [55] X. Ma, M. Xu, Q. Li, Y. Li, A. Zhou, and S. Wang, "Edge workload prediction based on deep learning," in *5G Edge Computing: Technologies, Applications and Future Visions*. Singapore: Springer, 2024, pp. 45–61.
- [56] G. E. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, *Time Series Analysis: Forecasting and Control*. Hoboken, NJ, USA: Wiley, 2015.
- [57] R. Zhao, D. Wang, R. Yan, K. Mao, F. Shen, and J. Wang, "Machine health monitoring using local feature-based gated recurrent unit networks," *IEEE Trans. Ind. Electron.*, vol. 65, no. 2, pp. 1539–1548, Feb. 2018.
- [58] W. Miao et al., "Workload prediction in edge computing based on graph neural network," in *Proc. IEEE ISPA/BDCloud/SocialCom/SustainCom*, 2021, pp. 1663–1666.
- [59] C. Que and F. N. Khan, "A scalable federated learning-based approach for accurate traffic prediction in edge computing-enabled metro optical networks," *Comput. Ind. Eng.*, vol. 203, May 2025, Art. no. 111004.
- [60] S. Kalra, J. Wen, J. C. Cresswell, M. Volkovs, and H. R. Tizhoosh, "Decentralized federated learning through proxy model sharing," *Nat. Commun.*, vol. 14, no. 1, p. 2899, 2023.
- [61] T. Wink and Z. Nocht, "An approach for peer-to-peer federated learning," in *Proc. IEEE/IFIP DSN Workshops*, 2021, pp. 150–157.
- [62] M. A. Dinani, A. Holzer, H. Nguyen, M. A. Marsan, and G. Rizzo, "Gossip learning of personalized models for vehicle trajectory prediction," in *Proc. IEEE WCNC Workshops*, 2021, pp. 1–7.
- [63] L. Palmieri, L. Valerio, C. Boldrini, and A. Passarella, "The effect of network topologies on fully decentralized learning: A preliminary investigation," in *Proc. NetAISys*, 2023, pp. 1–6.